

Lecture 2 Algorithm Analysis

Algorithm

a **abstract representation** of **limit sequence** of instructions used to solve some problems.
Program is the specific performance of algorithm.

Algorithm Analysis

- **Running time**
- **Memory required**
- Length of the program(lines of code)
- Ease of programming(bugs, maintenance)

An algorithm can run differently depending on

- the hardware platform
- the programming language
- **the scale of the problem**

so we need a **function of input size** to reflect the performance of the algorithm. That is **time/space complexity**.

Assumptions

- instructions are executed **sequentially**
- each instruction is **simple** and take exactly **one time unit**.
- integer size is fixed and we have **infinite memory**.

The point of counting the steps is to predict the **growth rate** in run time as the n scales up.

Asymptotic Notation

- Big-O, "bounded above by": $T(n) = O(f(n))$.
 - for some c and n_0 , $T(n) \leq cf(n)$ whenever $n > n_0$.
- Big-Omega, "bounded below by": $T(n) = \Omega(g(n))$
- Big-Theta, "bounded above and below": $T(n) = \Theta(h(n))$
- Little-o, "strictly bounded above"

Big-O Notation(Common Complexities)

constant time, logarithmic, linear, loglinear, square, cubic, exponential, factorial.

$(O(1), O(\log n), O(n), O(n \log n), O(n^2), O(n^3), O(2^n), O(n!))$

Useful Rules of Big-O Estimation

- Rule of sums

$$T_1(n) + T_2(n) = O(\max\{f(n), g(n)\})$$

- Rule of products

$$T_1(n) * T_2(n) = O(f(n) * g(n))$$

- $\log^k n = O(n)$ for any constant k . That is, logarithm increases very slowly

Simplification Methods

For polynomials whose sum is a constant number of terms, the function of big-O is n^k where k is the largest exponent in the polynomials.

Common Formula

- $\sum_{i=1}^n i = n(n+1)/2 = O(n^2)$
- $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6 = O(n^3)$
- $\sum_{i=1}^n x^i = (x^{n+1} - 1)(x - 1) = O(x^n)$

General rules of time complexity analysis

- simple statements(read, write, assign): constant $O(1)$
- simple operation (+ - and so on): constant $O(1)$
- sequence of simple statement or operations: depends on the most expensive operation.
- For, do and while loops
 - depends on the number of iterations.
 - nested loops: the products of the size of all loops
 - Example:

```
for (i = 1; i <= n; i++){
    for (j = 1; j <= i; j++){
        for (k = 1; k <= j; k++){
            operations;
        }
    }
}
```

$$\sum_{i=1}^n 1 + \dots + i = O(n^3)$$

```
int y = 5;
while ((y+1) * (y+1) < 5){
```

```
y++;  
}
```

the terminal condition is that $y \geq \sqrt{n} - 5$, thus the time complexity is $O(\sqrt{n})$

- if/else:

the judgment is constant $O(1)$, and the next time complexity depends on the bigger one of executive phase of if or else.

- method calls

A calls B, B calls C depends on the largest phase of them. The total running time is the sum of them.

Exercise

```
void func(int n, int arr[]) {  
    int m = 2^n ;  
    for (int i = 1; i < m; i *= 2) {  
        for (int j = 0; j < i; j++) {  
            printf("%d ", arr[j]);  
        }  
    }  
}
```

A $O(n^2)$

B $O(2^n)$

C $O(4^n)$

D $O(n \cdot 2^n)$

The total amount is $1 + 2 + 2^2 + \dots + 2^n = O(2^{n+1})$

Assume the recursive relation of a certain algorithm is :

$$T(n) = 2T(n/2) + n \log n \text{ (with } T(1) = 1\text{)}.$$

The time complexity of this algorithm is:

- | | |
|---|---|
| ☐ A $O(n \log n)$ | ☐ B $O(n^2 \log n)$ |
| ☐ C $O(n^2 (\log n)^2)$ | ☐ D $O(n (\log n)^2)$ |

Let $k = \log n$, then

$$T(n) = 2^k T(1) + n (\log n + \log \frac{n}{2} + \dots + \log 1) = n + n(\log n)^2 = O(n(\log n)^2)$$

```
void process(int n) {  
    int count = 0;  
    for (int i = n; i > 0; i /= 2) {  
        for (int j = 0; j < i; j++) {  
            count++;  
        }  
    }  
}
```

- | | |
|----------------------------------|---------------------------------------|
| ☐ A $O(n^2)$ | ☐ B $O(n \log n)$ |
| ☐ C $O(n)$ | ☐ D $O(\log n)$ |

$$n + \frac{n}{2} + \frac{n}{2^2} + \dots + \frac{n}{2^k} = 2n + C = O(n)$$

Checking Your Analysis (I)

Method 1

- when $T(N) = O(N)$, check if $T(2N)/T(N) = 2$

- when $T(N) = O(2N)$, check if $T(2N)/T(N) = 4$

Method 2

when $T(N) = O(f(N))$, check if $\frac{T(N)}{f(N)} = C$ for a range of N .

Max Subsequence Problem

- Method of Exhaustion
- Division-Conquer Method
- Dynamic Programming

Example: Computing X^N

we can divide the situation into N is even or odd.

```
long int Pow(long int x, unsigned int N){

    if (N == 0)

        return 1;

    if (N % 2 == 0){

        return Pow(x * x, N / 2);

    }else{

        return Pow(x * x, N / 2) * x;

    }

}
```

Note that we cannot use `return Pow(x, N/2) * Pow(x, N - N/2)` since it use two recursions and its time complexity is still $O(n)$.

The time complexity is $O(\log n)$.